# DATA ENGINEERING ON CLOUD

## E-commerce Data Integration: Analyse et gestion des Ventes et des Retours



**Présenté par** :

Pape Belele DIA

Ndoumbé SOW

**Professeur :**

Alexandre BERGERE

20 JULLIET 2024

**Tables des matières**

Présentation du Sujet ..... 3
Contexte ..... 3
Objectif ..... 3
Dataset ..... 3
Outils et Technologies utilisés ..... 4
Architecture ..... 5
Détails ..... 5
Modèle dimensionnel ..... 6
Schéma en étoile ..... 6
Implémentation ..... 7
Traitements et transformations ..... 7
ETL ..... 7
Création du Workspace ..... 7
Création du lakehouse ..... 8
Chargement des données ..... 8
Chargement de la table bronze ..... 13
Modèle dimensionnel ..... 16
Tables de dimension ..... 16
Table de Fait ..... 20
Workflow ..... 22
Automatisation ..... 23
Modèle Sémantique ..... 24
Visualisation ..... 26
Conclusion ..... 28
Sources : ..... 29

# Présentation du Sujet

## Contexte

De nos jours Les consommateurs se tournent de plus en plus vers les plateformes numériques pour effectuer leurs achats ce qui entraine sans nul doute l'ascension rapide de la vente en ligne au cours de la dernière décennie. Cette tendance s'est accentuée avec les récents développements technologiques.

Dans ce contexte dynamique, la gestion efficace des ventes et des retours est devenue cruciale pour maintenir la satisfaction des clients et optimiser leurs opérations. En effet cela permettra de suivre les commandes passées mais également d'avoir une vue d'ensemble sur les retours et de les gérer de manière efficace pour minimiser les pertes et comprendre les tendances clients.

## Objectif

L'objectif principal de cette étude est d'**offrir une meilleure expérience client en gérant efficacement les ventes et les retours, en réduisant les délais de traitement et en améliorant la communication avec les clients.**

Cependant cet objectif se décline en plusieurs sous-objectifs :

- Faciliter la prise de décision en fournissant des informations claires et précises
- Minimiser les pertes dues aux retours
- Analyser les raisons des retours pour identifier les produits problématiques ou les processus à améliorer, réduisant ainsi les coûts associés aux retours.
- Mettre en place des indicateurs de performance pour booster le commerce

Tout ceci sera basé sur les données collectées de nos différentes sources de données.

## Dataset

Dans notre solution nous allons utiliser deux fichiers csv qui seront nos sources de données.






Dans chaque fichier il y a deux rubriques : 

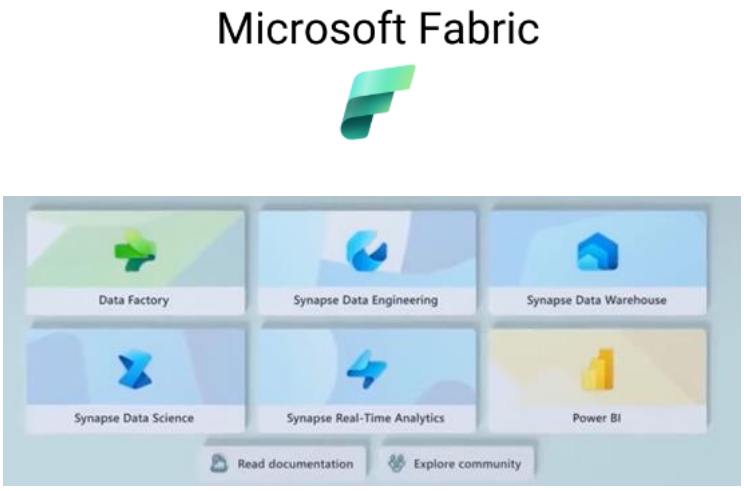

◊ "**Sales**" contient des informations sur :

- Les **commandes** (Order_ID, Order_Date, Shipping_Date, Aging, Ship_Mode, Shipping_Cost, Order_Priority)
- Les **produits** (Product_Category, Product)
- Les **ventes** (Sales, Quantity, Discount, Profit)
- Les **clients** (Customer_ID, Customer_Name, Segment, City, State,Country, Region)

◊ "**Returns**" contient des informations sur :

- Les **commandes** (Order_ID)
- Les **ventes** (Sales_Amount)
- Les **clients** (Customer_Name)
- Les **retours** (Return)

## Outils et Technologies utilisés




**Microsoft Fabric** est une plateforme de données et d'analytique complète, idéale pour unifier sa solution. Elle couvre le déplacement, l'ingestion, la transformation, le traitement des données en temps réel, et la génération de rapports. Elle propose des services intégrés tels que l'**ingénierie des données**, la **science des données**, l'**analytique en temps réel**, et les **entrepôts de données**. [1]

Dans notre projet nous avons orienté notre choix vers cette plateforme pour plusieurs raisons :

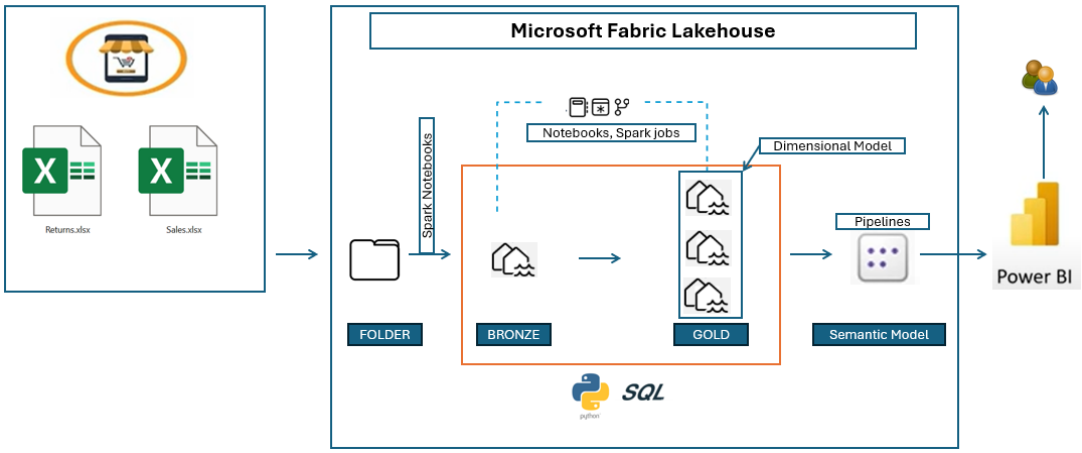
- Sa solution **Tout-en-Un** : la centralisation des opérations sur une seule plateforme, réduisant ainsi la complexité et élimine le besoin d'assembler des services de plusieurs fournisseurs

- Sa simplicité et sa Convivialité : une interface conviviale qui simplifie la gestion des données et des analyses

- Son modèle (SaaS) : les mises à jour et la maintenance sont gérées par Microsoft, ce qui va nous permettre de nous concentrer sur nos activités principales

- Sa sécurité et sa conformité : des niveaux élevés de sécurité et de conformité, garantissant que les données sont protégées contre les accès non autorisés et respectent les réglementations en vigueur

## Architecture




Détails :

- Dans notre solution, nous avons deux fichiers Excel que nous allons charger dans un dossier du Lakehouse que nous allons créer.

- A l'aide de notebooks Spark, nous allons intégrer ces données dans la "BRONZE LAYER" au sein d'une seule table regroupant toutes les informations relatives aux ventes (Sales) et aux retours (Return). Dans notre cas ceci nous permettra de stocker les données telles qu'elles sont reçues, sans transformation ni nettoyage significatif. Le plus important est de conserver un historique complet des données brutes, permettant de revenir en arrière (en cas de problème dans les étapes de transformation ultérieures) et d'examiner les données originales si nécessaire.

- Le "GOLD LAYER" est l'endroit où nous allons créer notre modèle dimensionnel (modèle en étoile), en structurant les données en une table de fait et des tables de dimensions pour faciliter les analyses.

- Nous établirons par la suite notre modèle sémantique qui nous sert à fournir une couche d'abstraction qui simplifie et enrichit l'accès aux données pour les utilisateurs finaux.il centralise les mesures, les calculs et les KPI (indicateurs clés de performance), garantissant que tous les utilisateurs et les rapports utilisent les mêmes définitions et calculs cohérents.

- Enfin, nous connecterons ce modèle sémantique à **Power BI** pour l'analyse et la visualisation des données.

Ce modèle sémantique structuré dans Microsoft Fabric, associé à Power BI, permet de créer des rapports interactifs et des tableaux de bord efficaces, offrant ainsi une vue complète et cohérente des opérations de vente et des retours.

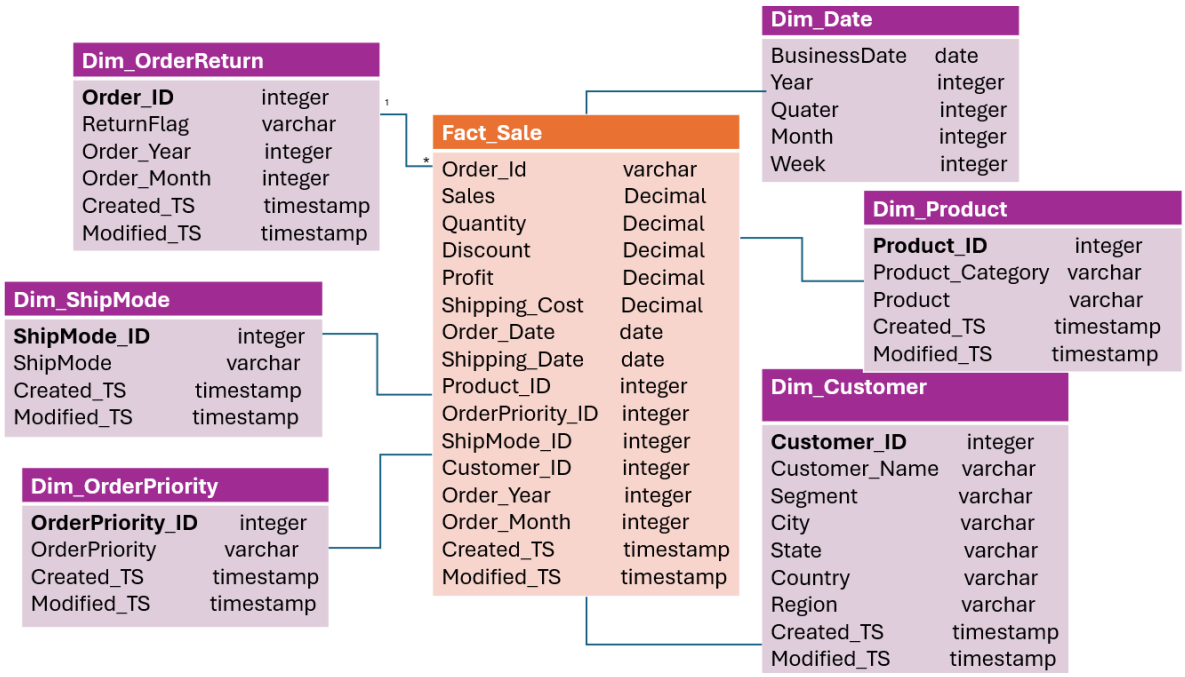
# Modèle dimensionnel

## Schéma en étoile

Nous avons fait le choix de faire un modèle en étoile car :

- Il est simple et facile à comprendre. Il se compose d'une table de faits centrale entourée de tables de dimensions.

- Les tables de dimensions sont généralement dénormalisées, ce qui permet d'accéder rapidement aux attributs nécessaires pour les analyses, réduisant ainsi le nombre de jointures complexes nécessaires.



Dans notre modèle dimensionnel, la table de fait **Fact_Sale** est entourée par les six tables de dimensions, créant une structure en étoile.

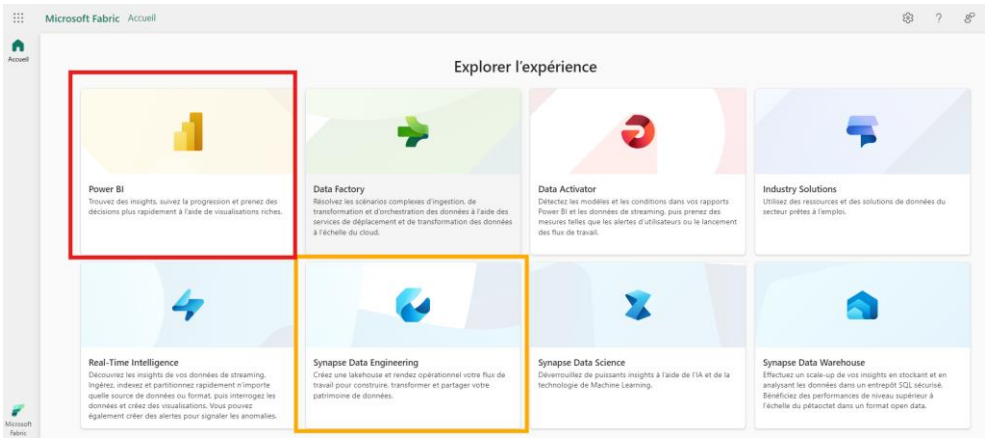
Chaque dimension apporte un contexte supplémentaire aux données de vente, permettant des analyses plus approfondies et des rapports détaillés.

# Implémentation

## Traitements et transformations

**Lien GitHub : https://github.com/ndoumbesow10/Data_Eng_OnCloud**

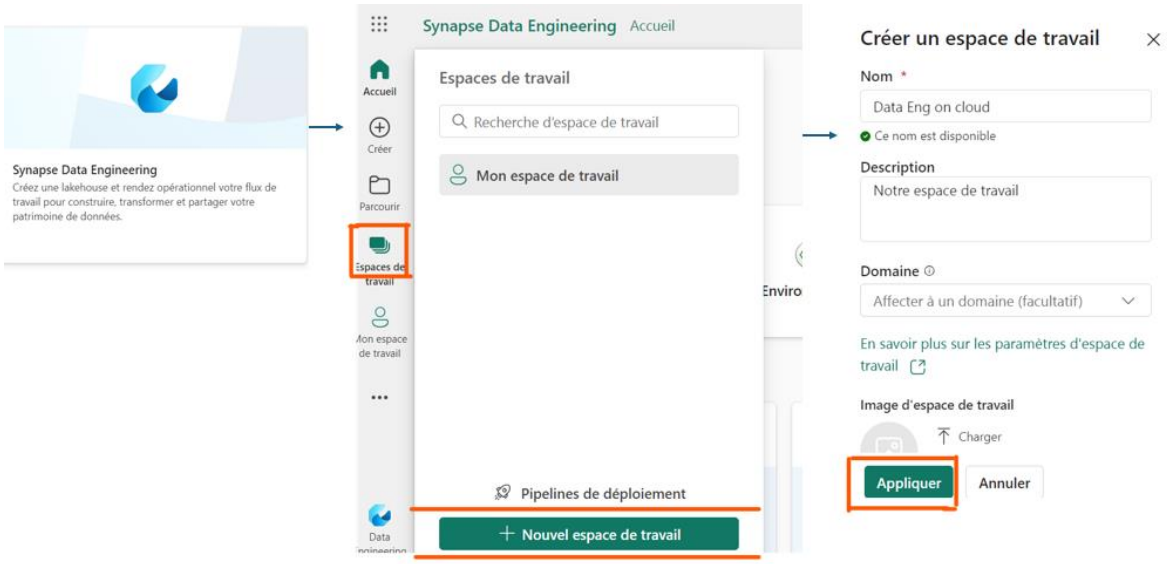
Dans notre démarche nous utilisons deux services de Microsoft Fabric, l'un pour la partie ETL et l'autre pour la partie visualisation.




## ETL

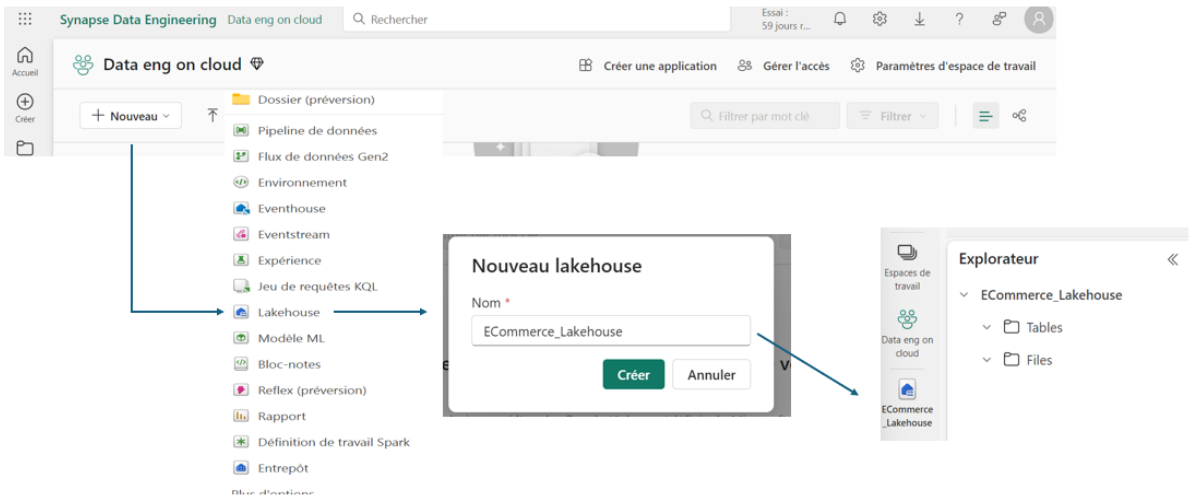
### *Création du Workspace*

Dans un premier temps nous créons un Workspace afin de collaborer sur le même projet en binôme, d'avoir une bonne structuration de notre projet et d'optimiser l'utilisation des ressources.



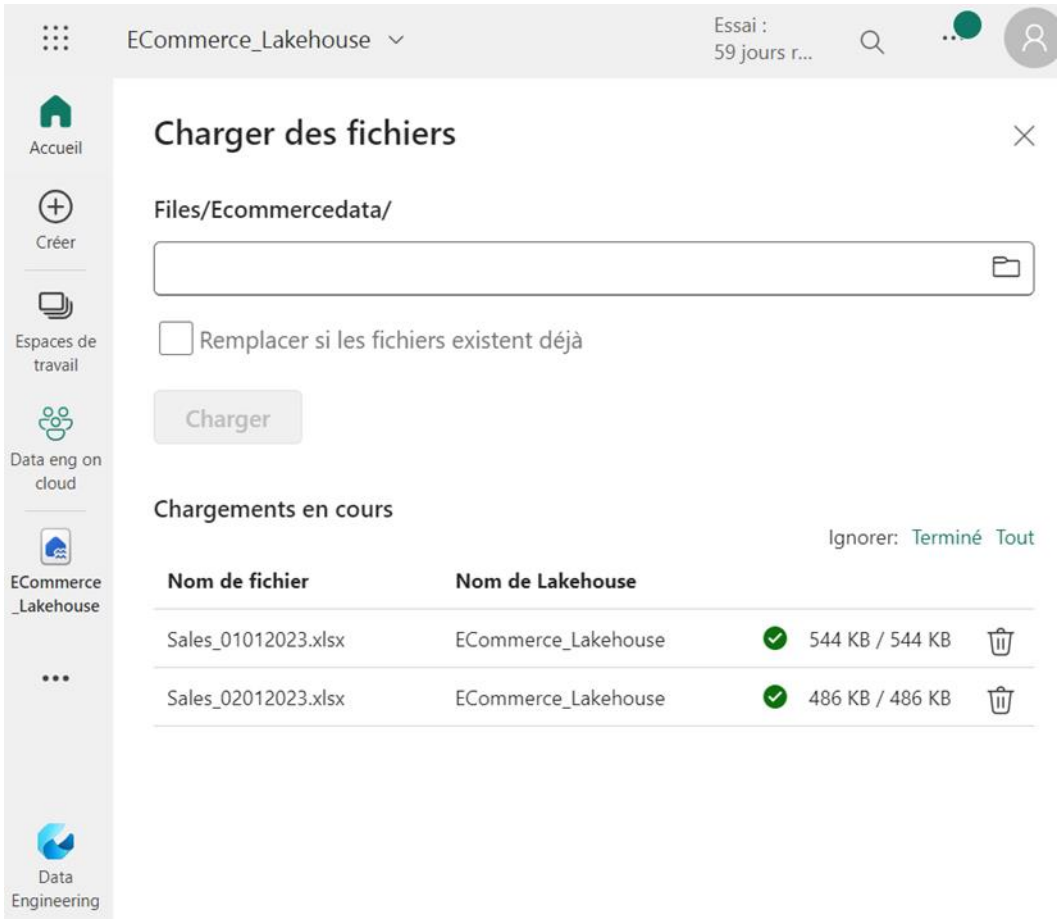
### *Création du lakehouse*

Ensuite nous créons notre Lakehouse.

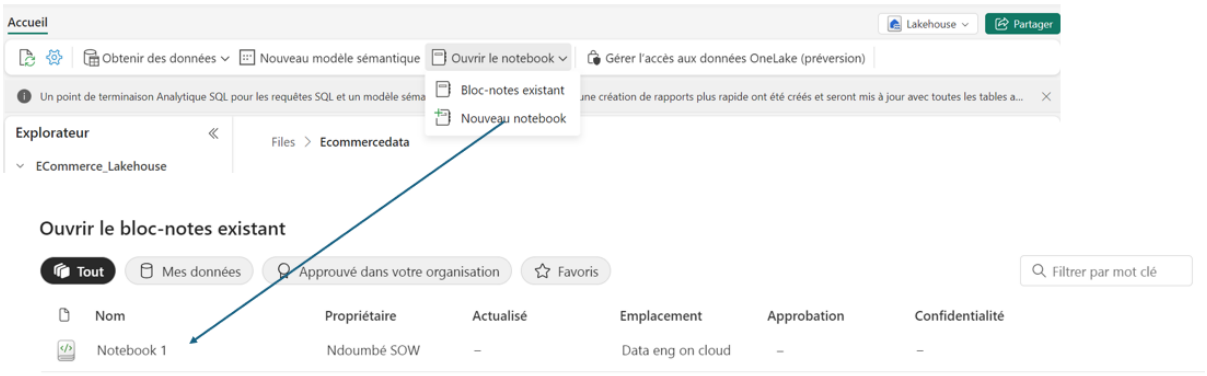



### *Chargement des données*

Pour le chargement des données nous avons créé dans “files” un sous-dossier “Ecommercedata” dans lequel nous avons chargé les deux fichiers qui nous servent de source de données.



Nous allons créer des notebooks pour chaque table crée dans notre solution afin d'avoir les détails de chacun et de faciliter l'analyse de la création de ces dites tables en cas de problème ou si nous voulons retourner en arrière pour une correction ou une vérification. Chaque notebook est renommé pour plus de précision et de clarté.




Comme précisé dans les détails de notre Architecture, nous avons deux sources de données et dans chacune se trouve des données de ventes et de retour.

Notre premier objectif est de centraliser toute ces données dans une même table.

Pour cela, nous avons besoins de certaines bibliothèques.

```
#import
from pyspark.sql import *
from pyspark.sql.functions import *
from pyspark.sql.types import *
import pandas as pd
```

✓ 14 s - La session Apache Spark sera prête dans 10 s 993 ms. Command executed in 3 s 144 ms by Ndoumbé SOW on · PySpark (Python)

Ici nous allons lire chaque feuille dans le but d'avoir un aperçu sur les données que nous avons pour les centraliser d'une part et prévoir les traitements qu'il faut d'une autre part.

Cette lecture se fera en créant un dataframe spark pour "Sales" et un dataFrame pour "Returns".

**Sales**

```
#Sales
df_sales= pd.read_excel("abfss://d36676f9-ba3b-477f-9cf2-998cd4d713d0@onelake.dfs.fabric.micro
#créer spark dataframe
df1=spark.createDataFrame(df_sales)
display(df1.head(5))
```

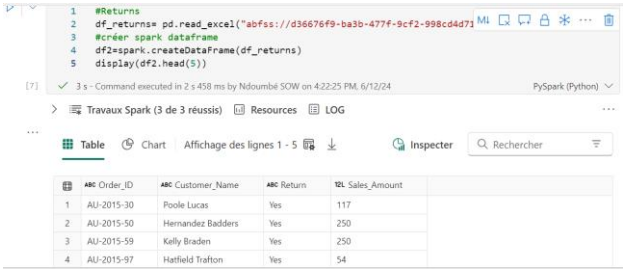
4 s - Command executed in 3 s 518 ms by Ndoumbé SOW on 4:15:13 PM, 6/12/24 PySpark (Python)

Travaux Spark (3 de 3 réussis) Resources LOG

Table Chart Affichage des lignes 1 - 5 Inspecter Rechercher

| | Order_ID | Order_Date | Shipping_Date | Aging | Ship_Mode | Product_Category | AB |
|---|---|---|---|---|---|---|---|
| 1 | AU-2015-30 | 2023-01-23 00:00... | 2023-01-27 00:00:00 | 4 | First Class | Auto & Accessories | C |
| 2 | AU-2015-50 | 2023-01-25 00:00... | 2023-01-30 00:00:00 | 5 | First Class | Auto & Accessories | T |
| 3 | AU-2015-59 | 2023-01-06 00:00... | 2023-01-12 00:00:00 | 6 | First Class | Auto & Accessories | T |
| 4 | AU-2015-81 | 2023-01-19 00:00... | 2023-01-27 00:00:00 | 8 | First Class | Auto & Accessories | C |

**Returns**



Une fois notre lecture faite sur les données nous pouvons joindre les deux dataframes.

```
#join datatafranes

df=df1.join(df2,df1.Order_ID==df2.Order_ID,how="left")
display(df.head(5))
```

[9] 2 s - Command executed in 1 s 563 ms by Ndoumbé SOW on 4:33:33 PM, 6/12/24 PySpark (Python)

Travaux Spark (4 de 4 réussis) Resources LOG

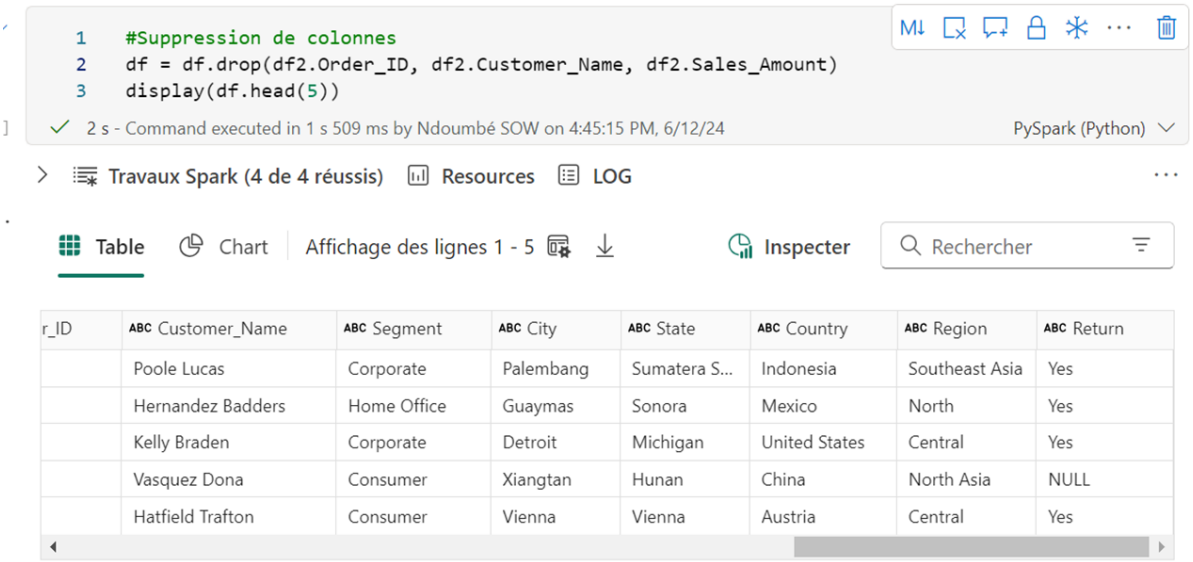
Table Chart Affichage des lignes 1 - 5 Inspecter Rechercher

| | State | Country | Region | Order_ID | Customer_Name | Return | Sales_Amount |
|---|---|---|---|---|---|---|---|
| bang | Sumatera S... | Indonesia | Southeast Asia | AU-2015-30 | Poole Lucas | Yes | 117 |
| has | Sonora | Mexico | North | AU-2015-50 | Hernandez Badders | Yes | 250 |
| t | Michigan | United States | Central | AU-2015-59 | Kelly Braden | Yes | 250 |
| an | Hunan | China | North Asia | NULL | NULL | NULL | NULL |
| | Vienna | Austria | Central | AU-2015-97 | Hatfield Trafton | Yes | 54 |

Cette jointure implique des colonnes identiques présentent deux fois, nous allons donc les supprimer pour éviter les doublons.

Les colonnes **Order_ID, Customer_Name** et **Sales_Amount** sont à supprimer car présentes dans les deux dataFrames que nous avons joint.

L'affichage du dataFrame montre que ces colonnes ont bien été supprimées du dataFrame.



```
#Suppression de colonnes
df = df.drop(df2.Order_ID, df2.Customer_Name, df2.Sales_Amount)
display(df.head(5))
```

2 s - Command executed in 1 s 509 ms by Ndoumbé SOW on 4:45:15 PM, 6/12/24 PySpark (Python)

Travaux Spark (4 de 4 réussis) Resources LOG

Table Chart Affichage des lignes 1 - 5 Inspecter Rechercher

| r_ID | Customer_Name | Segment | City | State | Country | Region | Return |
|---|---|---|---|---|---|---|---|
| | Poole Lucas | Corporate | Palembang | Sumatera S... | Indonesia | Southeast Asia | Yes |
| | Hernandez Badders | Home Office | Guaymas | Sonora | Mexico | North | Yes |
| | Kelly Braden | Corporate | Detroit | Michigan | United States | Central | Yes |
| | Vasquez Dona | Consumer | Xiangtan | Hunan | China | North Asia | NULL |
| | Hatfield Trafton | Consumer | Vienna | Vienna | Austria | Central | Yes |

Nous allons créer de nouveaux champs à notre table.

Pour la date :

- "Order_Year" : year("Order_Date") -> Crée une nouvelle colonne Order_Year en extrayant l'année de la colonne Order_Date.

- "Order_Month" : month("Order_Date") -> Crée une nouvelle colonne Order_Month en extrayant le mois de la colonne Order_Date.

La création de ces deux colonnes facilite l'analyse des données par année et par mois, ce qui est essentiel pour des rapports périodiques, des tendances de vente saisonnières, et des analyses de performance à long terme.

Pour le suivi :

- "Created_TS" : current_timestamp() -> Crée une nouvelle colonne Created_TS avec le timestamp de création de la ligne (au moment de l'exécution du code).

- "Modified_TS" : current_timestamp() -> Crée une nouvelle colonne Modified_TS avec le timestamp modification de la ligne (au moment de l'exécution du code).

Ceci permet d'avoir un suivi des changements c'est-à-dire suivre quand chaque ligne a été créée et modifiée.

Aide également à la gestion des versions et à la mise à jour des données, en fournissant des timestamps pour la gestion des modifications.

```
#Création de nouvelles colonnes mois,année,Created_TS et Modified_TS
df_modified=df.withColumns({"Order_Year": year("Order_Date"),\
            "Order_Month": month("Order_Date"),\
            "Created_TS": current_timestamp(),\
            "Modified_TS": current_timestamp(),\
            })
```

✓ < 1 s - Command executed in 267 ms by Ndoumbé SOW on 4:10:28 PM, 6/25/24

Avant la création de notre table, nous allons d'abord créer une vue temporaire qui va permettre de consolider et préparer les données.

Ceci est une étape clé dans le pipeline de traitement des données qui permet de simplifier et d'organiser l'accès aux données transformées via des requêtes SQL, facilitant ainsi la création de tables et la réalisation d'analyses complexes.

```
#Création d'une vue (dans le but de l'utiliser dans notre bronze table)

df_modified.createOrReplaceTempView("ViewSales")
```

✓ < 1 s - Command executed in 236 ms by Ndoumbé SOW on 4:10:29 PM, 6/25/24

Une fois la vue créée, elle est utilisée pour créer la table bronze.

```
%%sql
create table if not exists ECommerce_Lakehouse.Bronze_Sales
(
    Order_ID    string,
    Order_Date  Date,
    Shipping_Date   date,
    Aging   int,
    Ship_Mode   string,
    Product_Category    string,
    Product  string,
    Sales   float,
    Quantity    float,
    Discount     float,
    Profit   float,
    Shipping_Cost   float,
    Order_Priority  string,
    Customer_ID string,
    Customer_Name   string,
    Segment string,
    City    string,
    State   string,
    Country string,
    Region string,
    Return string,
    Order_Year int,
    Order_Month int,
    Created_TS  TIMESTAMP,
    Modified_TS  TIMESTAMP
```

Cependant lors de la création de la table bronze, nous utilisons **using DELTA** qui indique que la table sera stockée en utilisant le format Delta Lake. Delta Lake est une couche de stockage qui apporte fiabilité et performance au datalake.

Le partitionnement par **Order_Year** et **Order_Month** permet d'améliorer les performances des requêtes qui filtrent les données par année et mois de commande. Spark lira uniquement les partitions pertinentes, réduisant ainsi le volume de données à traiter.

```
)
using DELTA
PARTITIONED by(Order_Year,Order_Month)
```

### *Chargement de la table bronze*

Pour charger cette table, le script “**MERGE**” qui permet de maintenir la table dans un état synchronisé avec les données de la vue temporaire créée précédemment.

Dans le cas où les enregistrements de BS et VS correspondent, les colonnes de BS sont mises à jour avec les valeurs de VS (vue).

```
--Insertion des données dans bronze
Merge into  ECommerce_Lakehouse.Bronze_Sales as BS
using ViewSales as VS
on  BS.Order_Year=VS.Order_Year and BS.Order_Month=VS.Order_Month and BS.Order_ID=VS.Order_ID
when matched then
update SET
BS.Order_Date   =   VS.Order_Date   ,
BS.Shipping_Date    =   VS.Shipping_Date    ,
BS.Aging    =   VS.Aging    ,
BS.Ship_Mode    =   VS.Ship_Mode    ,
BS.Product_Category =   VS.Product_Category ,
BS.Product  =   VS.Product  ,
BS.Sales    =   VS.Sales    ,
BS.Quantity =   VS.Quantity ,
BS.Discount =   VS.Discount ,
BS.Profit   =   VS.Profit   ,
BS.Shipping_Cost    =   VS.Shipping_Cost    ,
BS.Order_Priority   =   VS.Order_Priority   ,
BS.Customer_ID  =   VS.Customer_ID  ,
BS.Customer_Name    =   VS.Customer_Name    ,
BS.Segment  =   VS.Segment  ,
BS.City =   VS.City ,
BS.State    =   VS.State    ,
BS.Country  =   VS.Country  ,
BS.Region   =   VS.Region   ,
BS.Return   =   VS.Return   ,
BS.Modified_TS  =   VS.Modified_TS
```

Dans le cas contraire, de nouveaux enregistrements sont insérés dans BS avec les valeurs de VS.

```
when not matched then
INSERT
(
BS.Order_ID,
BS.Order_Date   ,
BS.Shipping_Date    ,
BS.Aging    ,
BS.Ship_Mode    ,
BS.Product_Category ,
BS.Product  ,
BS.Sales    ,
BS.Quantity ,
BS.Discount ,
BS.Profit   ,
BS.Shipping_Cost    ,
BS.Order_Priority   ,
BS.Customer_ID  ,
BS.Customer_Name    ,
BS.Segment  ,
BS.City ,
BS.State    ,
BS.Country  ,
BS.Region   ,
BS.Return,
BS.Order_Year   ,
BS.Order_Month      ,
BS.Created_TS       ,
BS.Modified_TS
```

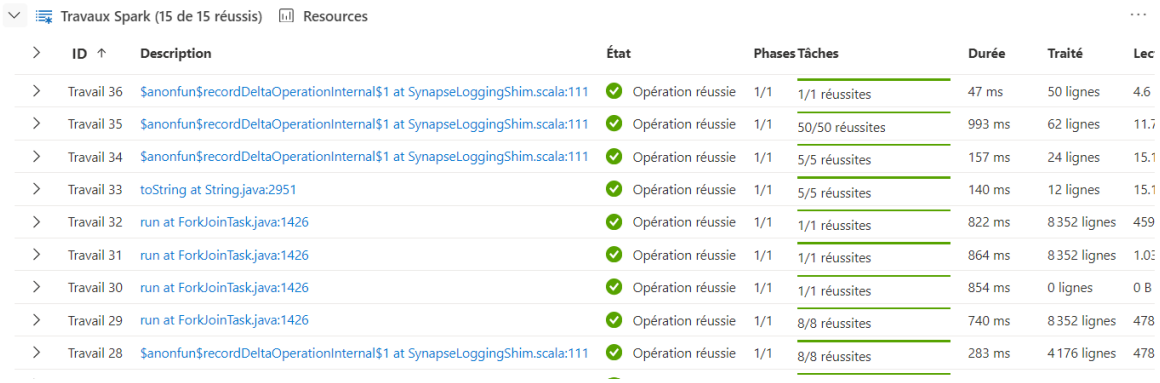
```
values
(
VS.Order_ID,
VS.Order_Date   ,
VS.Shipping_Date    ,
VS.Aging    ,
VS.Ship_Mode    ,
VS.Product_Category ,
VS.Product  ,
VS.Sales    ,
VS.Quantity ,
VS.Discount ,
VS.Profit   ,
VS.Shipping_Cost    ,
VS.Order_Priority   ,
VS.Customer_ID  ,
VS.Customer_Name    ,
VS.Segment  ,
VS.City ,
VS.State    ,
VS.Country  ,
VS.Region   ,
VS.Return,
VS.Order_Year   ,
VS.Order_Month      ,
VS.Created_TS       ,
VS.Modified_TS
)
```

En utilisant une clé de correspondance (Order_Year, Order_Month, Order_ID), cette opération assure que chaque enregistrement est unique et que les données ne sont pas dupliquées.

En effet dans le cas d'une correspondance lorsqu'un enregistrement dans la vue source (ViewSales) correspond à un enregistrement existant dans la table cible (Bronze_Sales) selon la clé définie (Order_Year, Order_Month, Order_ID), les données de cet enregistrement dans la table cible sont mises à jour avec les dernières valeurs de la vue source.

Et dans le cas d'une absence de correspondance chaque nouvel enregistrement dans la vue source est ajouté à la table cible uniquement s'il n'existe pas déjà selon cette clé de correspondance, évitant ainsi les duplications.

Les jobs spark montrent bien que les opérations effectuées sont terminées et ont réussi.



Travaux Spark (15 de 15 réussis) Resources

| ID | Description | État | Phases | Tâches | Durée | Traité | Lec |
|---|---|---|---|---|---|---|---|
| Travail 36 | $anonfun$recordDeltaOperationInternal$1 at SynapseLoggingShim.scala:111 | Opération réussie | 1/1 | 1/1 réussites | 47 ms | 50 lignes | 4.6 |
| Travail 35 | $anonfun$recordDeltaOperationInternal$1 at SynapseLoggingShim.scala:111 | Opération réussie | 1/1 | 50/50 réussites | 993 ms | 62 lignes | 11.7 |
| Travail 34 | $anonfun$recordDeltaOperationInternal$1 at SynapseLoggingShim.scala:111 | Opération réussie | 1/1 | 5/5 réussites | 157 ms | 24 lignes | 15.1 |
| Travail 33 | toString at String.java:2951 | Opération réussie | 1/1 | 5/5 réussites | 140 ms | 12 lignes | 15.1 |
| Travail 32 | run at ForkJoinTask.java:1426 | Opération réussie | 1/1 | 1/1 réussites | 822 ms | 8 352 lignes | 459 |
| Travail 31 | run at ForkJoinTask.java:1426 | Opération réussie | 1/1 | 1/1 réussites | 864 ms | 8 352 lignes | 1.03 |
| Travail 30 | run at ForkJoinTask.java:1426 | Opération réussie | 1/1 | 1/1 réussites | 854 ms | 0 lignes | 0 B |
| Travail 29 | run at ForkJoinTask.java:1426 | Opération réussie | 1/1 | 8/8 réussites | 740 ms | 8 352 lignes | 478 |
| Travail 28 | $anonfun$recordDeltaOperationInternal$1 at SynapseLoggingShim.scala:111 | Opération réussie | 1/1 | 8/8 réussites | 283 ms | 4 176 lignes | 478 |

Les données ont bien été insérées.

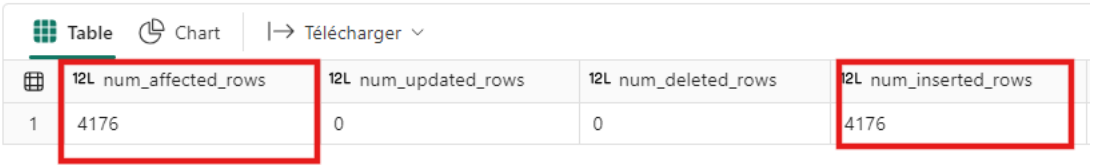
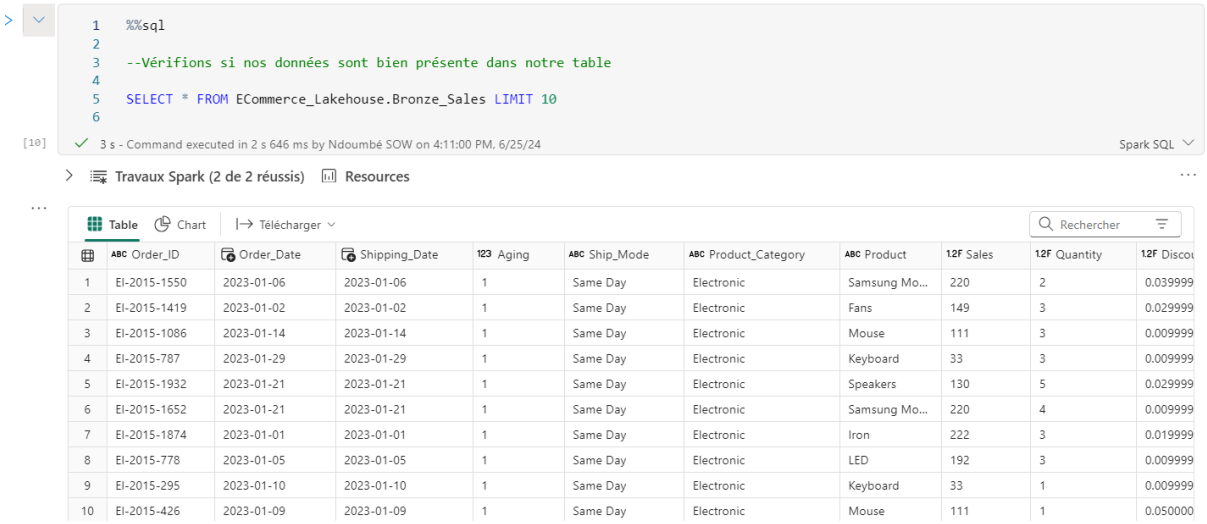


Table Chart Télécharger

| | num_affected_rows | num_updated_rows | num_deleted_rows | num_inserted_rows |
|---|---|---|---|---|
| 1 | 4176 | 0 | 0 | 4176 |



```
%%sql

--Vérifions si nos données sont bien présente dans notre table

SELECT * FROM ECommerce_Lakehouse.Bronze_Sales LIMIT 10
```

[10] 3 s - Command executed in 2 s 646 ms by Ndoumbé SOW on 4:11:00 PM, 6/25/24 Spark SQL

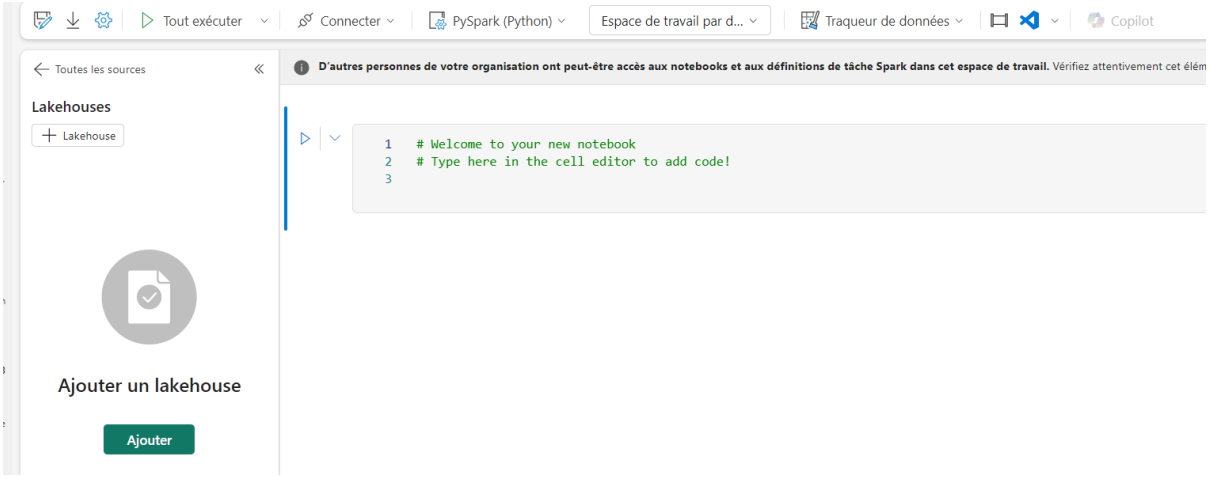
Travaux Spark (2 de 2 réussis) Resources

Table Chart Télécharger Rechercher

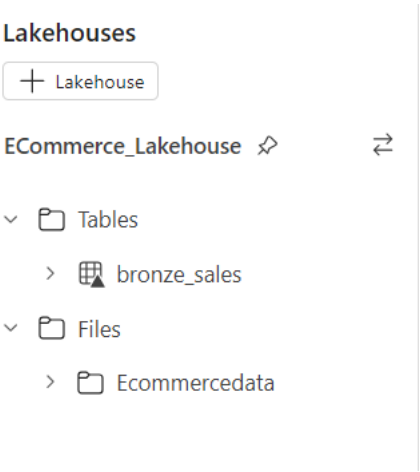
| | Order_ID | Order_Date | Shipping_Date | Aging | Ship_Mode | Product_Category | Product | Sales | Quantity | Disco |
|---|---|---|---|---|---|---|---|---|---|---|
| 1 | EI-2015-1550 | 2023-01-06 | 2023-01-06 | 1 | Same Day | Electronic | Samsung Mo... | 220 | 2 | 0.039999 |
| 2 | EI-2015-1419 | 2023-01-02 | 2023-01-02 | 1 | Same Day | Electronic | Fans | 149 | 3 | 0.029999 |
| 3 | EI-2015-1086 | 2023-01-14 | 2023-01-14 | 1 | Same Day | Electronic | Mouse | 111 | 3 | 0.009999 |
| 4 | EI-2015-787 | 2023-01-29 | 2023-01-29 | 1 | Same Day | Electronic | Keyboard | 33 | 3 | 0.009999 |
| 5 | EI-2015-1932 | 2023-01-21 | 2023-01-21 | 1 | Same Day | Electronic | Speakers | 130 | 5 | 0.029999 |
| 6 | EI-2015-1652 | 2023-01-21 | 2023-01-21 | 1 | Same Day | Electronic | Samsung Mo... | 220 | 4 | 0.009999 |
| 7 | EI-2015-1874 | 2023-01-01 | 2023-01-01 | 1 | Same Day | Electronic | Iron | 222 | 3 | 0.019999 |
| 8 | EI-2015-778 | 2023-01-05 | 2023-01-05 | 1 | Same Day | Electronic | LED | 192 | 3 | 0.009999 |
| 9 | EI-2015-295 | 2023-01-10 | 2023-01-10 | 1 | Same Day | Electronic | Keyboard | 33 | 1 | 0.009999 |
| 10 | EI-2015-426 | 2023-01-09 | 2023-01-09 | 1 | Same Day | Electronic | Mouse | 111 | 1 | 0.050000 |

### *Modèle dimensionnel*

Toujours dans le même espace de travail, la création de notebooks pour le model dimensionnel est faite.




Et on ajoute à notre Lakehouse existant.




Les “Gold_Layer” est l’endroit où le modèle dimensionnel se fera.

Pour chaque table de dimension et la table de fait un notebook est créé et les traitements faits dessus.

Les tables de dimension sont créées avant la table de fait car les clés étrangères de ces tables iront vers la table de fait.

### Tables de dimension

Exemple de création de tables de dimension :

- Dim_Customer

Tout d’abord la table est créée en spécifiant ses colonnes, toujours en utilisant le delta lake.

```
%%sql
create table if not exists ECommerce_Lakehouse.gold_Customer
(
  Customer_ID string  ,
Customer_Name string  ,
Segment string  ,
City  string  ,
State string  ,
Country string  ,
Region  string  ,
Created_TS  timestamp ,
Modified_TS timestamp

)
using delta
```

✓ - Command executed in 16 s 559 ms by Ndoumbé SOW on 5:50:20 PM, 6/23/24

Dans cette partie du code, la date de la dernière modification d'un enregistrement dans la table gold_Customer peut être obtenue.

```
Max_Date=spark.sql("""select coalesce(max(Modified_TS),"1900-01-01") from ECommerce_Lakehouse.gold_Customer""").first()[0]
```

max(Modified_TS) : Sélectionne la date maximale dans la colonne Modified_TS, représentant la date la plus récente de modification des enregistrements.

coalesce(max(Modified_TS), "1900-01-01") : Utilise la fonction coalesce pour remplacer les valeurs nulles par une date par défaut ("1900-01-01"). Cela signifie que si la table est vide et qu'il n'y a pas de valeurs dans Modified_TS, la date renvoyée sera "1900-01-01".

Cette action peut être très utile sur le long terme surtout. En connaissant la date de la dernière modification, des mécanismes d'archivage ou de versioning des données peuvent être mis en place. Cela permet de conserver des historiques précis des changements de données pour une analyse rétrospective. Par exemple, des changements dans les segments de clients peuvent déclencher des actions marketing ou de support client.

Les données qui doivent être chargées dans les tables de dimension et la table de fait proviennent de la table bronze.

```
df_bronze=spark.read.table("ECommerce_Lakehouse.Bronze_Sales")

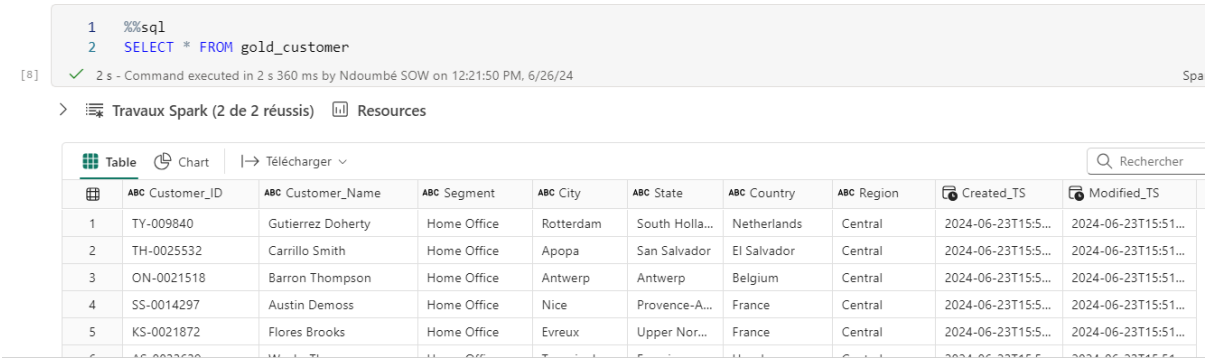
df=df_bronze.selectExpr("Customer_ID","Customer_Name","Segment","City","State", \
                  "Country","Region")\
                  .where(col("Modified_TS")>Max_Date)\
                  .drop_duplicates()
```

Ici les colonnes pertinentes pour les informations clients de la table “Bronze_Sales” sont sélectionnées, les enregistrements sont également filtrés en fonction de la dernière date de modification et les doublons sont supprimés pour garantir l'unicité des enregistrements. Ces étapes préparent efficacement les données, en vue de les charger dans la table gold_Customer.

Par la suite une vue est créée comme fait précédemment lors de la création de la table “Bronze_Sales” dans le Bronze layer.

```
df.createOrReplaceTempView("ViewCustomer")
```

Et enfin merger la vue et la table afin d’avoir les données de la vue dans la table.



```
%%sql
SELECT * FROM gold_customer
```

[8] ✓ 2 s - Command executed in 2 s 360 ms by Ndoumbé SOW on 12:21:50 PM, 6/26/24

Travaux Spark (2 de 2 réussis) Resources

Table Chart Télécharger Rechercher

| | Customer_ID | Customer_Name | Segment | City | State | Country | Region | Created_TS | Modified_TS |
|---|---|---|---|---|---|---|---|---|---|
| 1 | TY-009840 | Gutierrez Doherty | Home Office | Rotterdam | South Holla... | Netherlands | Central | 2024-06-23T15:5... | 2024-06-23T15:51... |
| 2 | TH-0025532 | Carrillo Smith | Home Office | Apopa | San Salvador | El Salvador | Central | 2024-06-23T15:5... | 2024-06-23T15:51... |
| 3 | ON-0021518 | Barron Thompson | Home Office | Antwerp | Antwerp | Belgium | Central | 2024-06-23T15:5... | 2024-06-23T15:51... |
| 4 | SS-0014297 | Austin Demoss | Home Office | Nice | Provence-A... | France | Central | 2024-06-23T15:5... | 2024-06-23T15:51... |
| 5 | KS-0021872 | Flores Brooks | Home Office | Evreux | Upper Nor... | France | Central | 2024-06-23T15:5... | 2024-06-23T15:51... |

La table gold_Customer est bien chargée.

- Dim_Date

Création de la table gold_date avec les champs et leur type.

```
%%sql
create table if not EXISTS ECommerce_Lakehouse.gold_date
(
BusinessDate Date,
Business_Year int,
Business_Month int,
Business_Quarter int,
Business_Week int
)
using DELTA
PARTITIONED by (Business_Year)
```

Ensuite nous calculons la différence en jour entre le 01 janvier 2015 et le 31 décembre 2030. Ceci permet de mieux segmenter les données par différentes périodes, telles que jours, semaines, mois, trimestres et années.

```
start_date='2015-01-01'
end_date='2030-12-31'

date_diff=spark.sql("select date_diff('{}','{}')".format(end_date,start_date)).first()[0]
```

A partir de cette différence, une séquence de nombre est créée. Cela permet, par exemple, de générer une table de dimension date où chaque ligne représente un jour unique dans la plage spécifiée.

Ensuite une colonne de dates basées sur une séquence d'ID est créée.

```
1  date_data=ID_Data.selectExpr("date_add('{}',cast(id as int)) as BusinessDate".format(start_date))
```

✓ < 1 s - Command executed in 236 ms by Ndoumbé SOW on 1:10:54 PM, 6/26/24

Ces actions ont plusieurs utilités :

- Permet d'analyser les données à travers différentes périodes (jours, semaines, mois, années).
- Aide à identifier les tendances, les cycles saisonniers, et les anomalies temporelles.
- Permet de créer des résumés temporels comme les ventes mensuelles, les visites quotidiennes, etc.

La ViewDate est créée ensuite pour préparer la création de la table de dimension comme fait précédemment.

```
date_data.createOrReplaceTempView("ViewDate")
```

Dans le contexte des tables de dimension, comme une table de dates, il est courant de régulièrement mettre à jour l'ensemble des dates pour couvrir de nouvelles périodes ou ajuster les attributs des dates existantes (par exemple, ajouter des jours fériés, ajuster les jours ouvrables, etc.).

Donc les données existantes dans notre table vont être supprimées avant d'insérer de nouvelles données.

```
%%sql
delete from ECommerce_Lakehouse.gold_date;
```

Ensuit les données de la vue temporaire **ViewDate** sont insérées dans la table **gold_date**.

```
%%sql
insert into ECommerce_Lakehouse.gold_date
select *,
Year(BusinessDate) as Business_Year,
Month(BusinessDate) as Business_Month,
Quarter(BusinessDate) as Business_Quarter,
weekday(BusinessDate) as Business_Week
from ViewDate
```

Les données sont bien insérées dans la dimension Date.

```
%%sql
SELECT count(*) FROM gold_date
```

[11] ✓ 2 s - Command executed in 1 s 544 ms by Ndoumbé SOW on 1:11:12 PM, 6/26/24

Travaux Spark (2 de 2 réussis) Resources LOG

Table Chart Télécharger

| | 12L count(1) |
|---|---|
| 1 | 5843 |

## Table de Fait

Nous avons dans notre solution une table de fait.

On commence par créer la table avec ses attributs.

```
%%sql
create table if not EXISTS ECommerce_Lakehouse.gold_fact_sale
(
Order_ID    string  ,
Price float,
Quantity    float   ,
Sales   float   ,
Discount    float   ,
Profit  float   ,
Shipping_Cost   float   ,
Order_Date  date    ,
Shipping_Date   date    ,
Product_ID  long    ,
OrderPriority_ID    Long    ,
ShipMode_ID Long    ,
Customer_ID string  ,
Order_Year  integer ,
Order_Month integer ,
Created_TS  timestamp   ,
Modified_TS timestamp
)
using DELTA
PARTITIONED by (Order_Year,Order_Month)
```

✓ - Command executed in 7 s 992 ms by Ndoumbé SOW on 8:50:35 PM, 6/23/24

Ensuite nous allons charger la table de fait.

La table de fait est utilisée pour enregistrer les mesures quantitatives des activités commerciales. Elle contient des données numériques et des clés étrangères qui se réfèrent aux tables de dimension que nous avons déjà créés.

Pour cela, des calculs et des jointures sont faits :

- **bronze_sales.Sales * bronze_sales.Quantity as Sales** : Calcul du montant total des ventes pour la commande.
- Jointures :

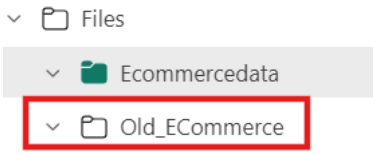
  - **Jointure avec gold_product** :
    - Permet d'obtenir l'identifiant unique du produit (Product_ID) en associant les colonnes Product et Product_Category de bronze_sales avec celles de gold_product.
  - **Jointure avec gold_shipmode** :

    - Permet d'obtenir l'identifiant unique du mode d'expédition (ShipMode_ID) en associant la colonne Ship_Mode de bronze_sales avec celle de gold_shipmode.
- **Jointure avec gold_orderpriority** :
    - Permet d'obtenir l'identifiant unique de la priorité de commande (OrderPriority_ID) en associant la colonne Order_Priority de bronze_sales avec celle de gold_orderpriority.

```
df_bronze= spark.sql("""select
bronze_sales.Order_ID   ,
bronze_sales.Sales as Price,
bronze_sales.Quantity,
bronze_sales.Sales  * bronze_sales.Quantity as Sales,
bronze_sales.Discount   ,
bronze_sales.Profit ,
bronze_sales.Shipping_Cost  ,
bronze_sales.Order_Date ,
bronze_sales.Shipping_Date  ,
gold_product.Product_ID ,
gold_orderpriority.OrderPriority_ID ,
gold_shipmode.ShipMode_ID   ,
bronze_sales.Customer_ID    ,
Year(Order_Date) as Order_Year  ,
Month(Order_Date) as Order_Month
from ECommerce_Lakehouse.bronze_sales
inner join ECommerce_Lakehouse.gold_product on bronze_sales.Product=gold_product.Product and
                                        bronze_sales.Product_Category=gold_product.Product_Category
inner join ECommerce_Lakehouse.gold_shipmode on bronze_sales.Ship_Mode= gold_shipmode.Ship_Mode
inner join ECommerce_Lakehouse.gold_orderpriority on bronze_sales.Order_Priority=gold_orderpriority.Order_Priority
""")
```

Ensuite comme pour les autres tables, une vue est créée et un merge est fait pour charger les données dans la table de fait.

Une fois que notre modèle dimensionnel est prêt, nous créons un deuxième répertoire.




Ainsi nous avons deux répertoires distincts, l'un pour les nouveaux fichiers et l'autre pour les anciens fichiers, ceci est une pratique courante dans les workflows de traitement de données.

Cette approche présente plusieurs avantages :

- En séparant les nouveaux fichiers des anciens, nous pouvons facilement distinguer les fichiers qui n'ont pas encore été traités de ceux qui l'ont été.

- Il est plus facile de s'assurer que les fichiers ne sont pas traités plusieurs fois par erreur.

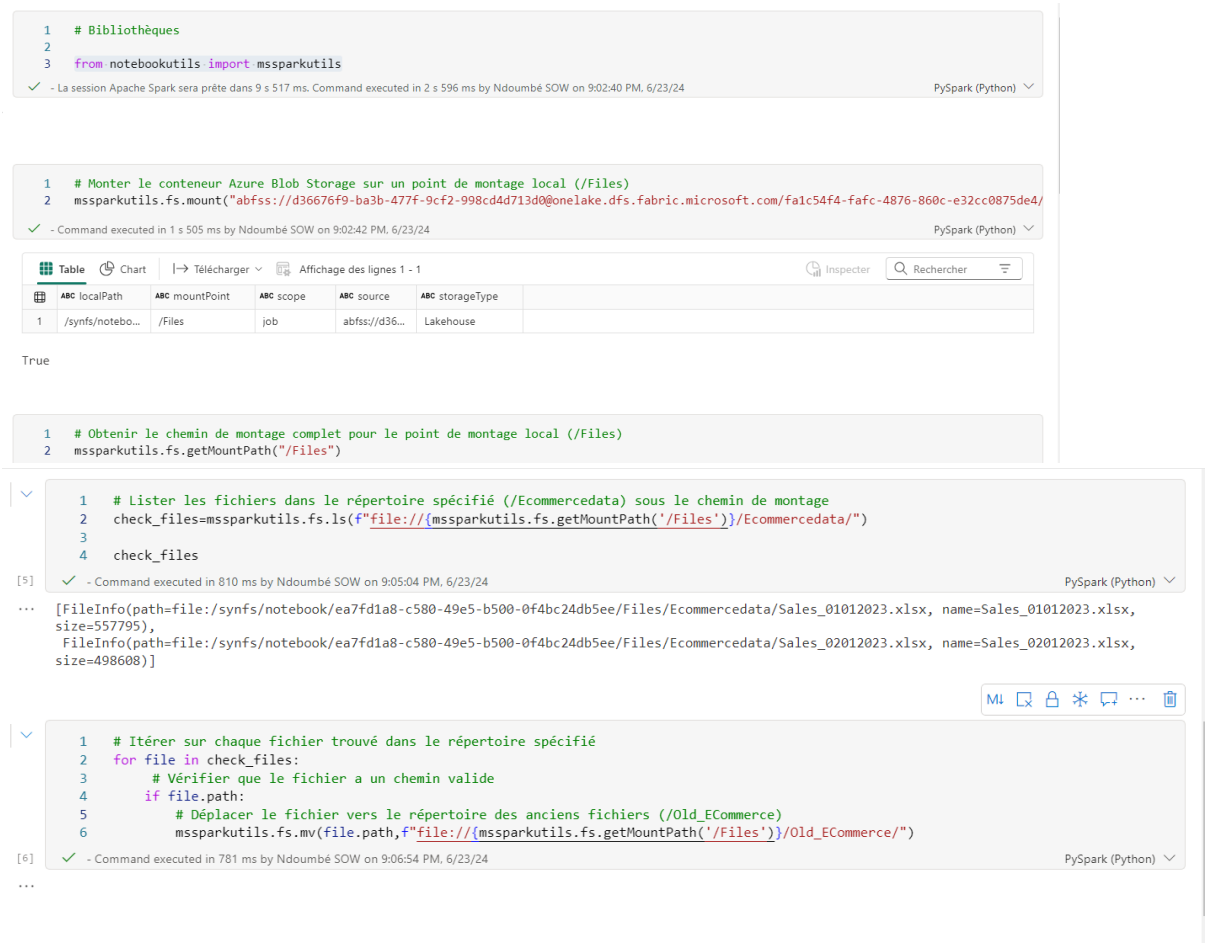
- Il permet de garder une trace des fichiers traités donc de conserver un historique des opérations.

- En cas de besoin d'audit, nous pouvons facilement accéder aux fichiers traités et vérifier quand et comment ils ont été manipulés.

- La gestion des erreurs de traitement se fait plus facilement.

Pour cela, un notebook est créé et avec nos scripts dedans.

On importe d'abord **mssparkutils**, qui est utilisé pour gérer les systèmes de fichiers et exécuter des notebooks dans un environnement Microsoft Fabric.




*Workflow*

1. **Nouveaux Fichiers** : Tous les nouveaux fichiers sont placés dans un répertoire dédié (e.g., /Ecommercedata/).

2. **Traitement** : des script ou jobs surveillent ce répertoire, traitent les fichiers un par un.

3. **Déplacement** : Une fois traités, les fichiers sont déplacés vers un répertoire d'anciens fichiers (e.g., /Old_ECommerce/).

4. **Vérification** : Le répertoire des anciens fichiers peut être régulièrement vérifié pour l'archivage ou la suppression des fichiers anciens, selon les besoins de rétention des données.

## Automatisation

Dans notre méthode de travail, nous avons choisi de créer un notebook pour chaque action/tables afin d'avoir le détail de chaque étape. Nous avons créé un notebook ( Run_Load) dans lequel nous avons appelé tous les autres notebooks créés précédemment.

Ceci nous permettra d'exécuter tous les notebooks listés.

```
1  if check_files:
2      mssparkutils.notebook.run("BronzeTable")
3      mssparkutils.notebook.run("Gold_Customer")
4      mssparkutils.notebook.run("Gold_Date")
5      mssparkutils.notebook.run("Gold_Product")
6      mssparkutils.notebook.run("Gold_ShipMode")
7      mssparkutils.notebook.run("Gold_OrderPriority")
8      mssparkutils.notebook.run("Gold_OrderReturn")
9      mssparkutils.notebook.run("Gold_Fact_Sale")
10     mssparkutils.notebook.run("Archives_Files")
✓ - Command executed in 241 ms by Ndoumbé SOW on 9:29:07 PM, 6/23/24
```

Dans ce notebook, nous avons listé les fichiers et exécuté les notebooks de traitement de données automatiquement afin de créer un pipeline de données efficace, fiable et extensible.

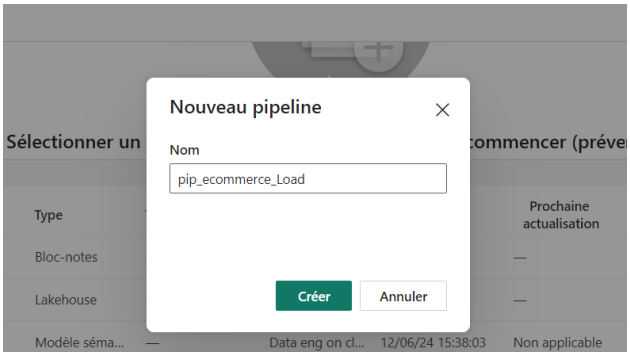
Cette procédure permet :

- D'automatiser le traitement des données pour éviter les interventions manuelles, économiser du temps et réduire les erreurs. Ainsi le pipeline de traitement peut fonctionner de manière autonome, traitant de nouveaux fichiers dès qu'ils sont disponibles.

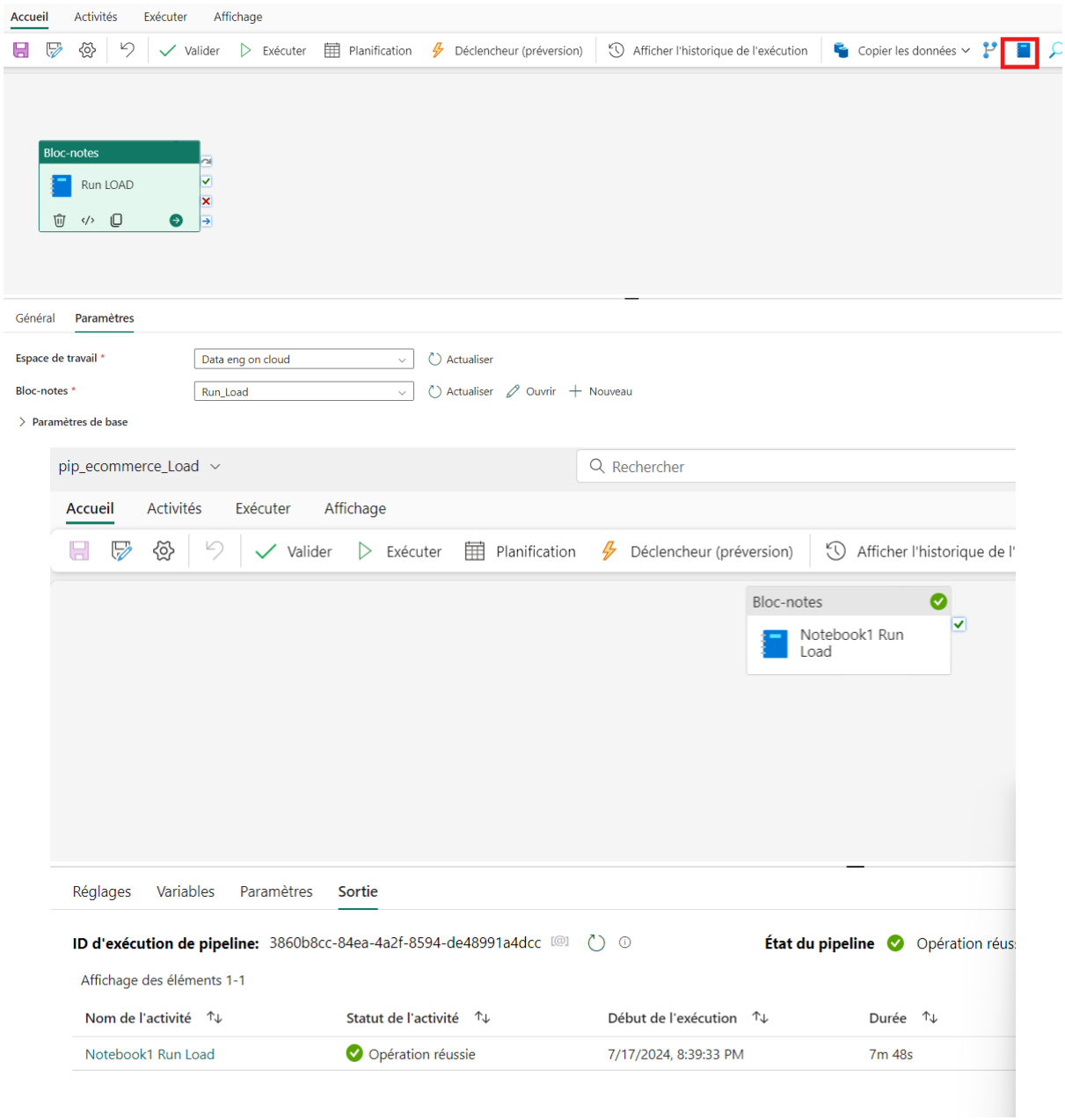
- D'organiser les données en différentes étapes de transformation (Bronze, et Gold dans notre cas) pour une meilleure gestion et traçabilité.

- D'ajouter ou de modifier les notebooks de traitement sans perturber le workflow global.

Nous avons par la suite créé un pipeline qui va nous permettre d'exécuter tous nos notebooks.



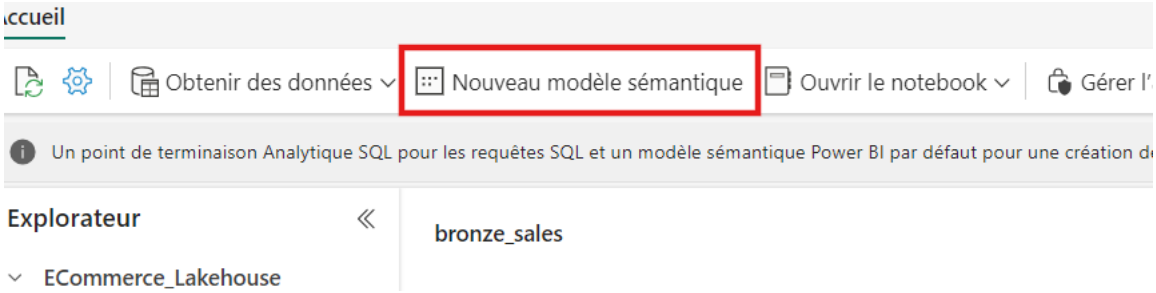
Un fois créé, dans les paramètres on lui indique le notebook qu'il doit exécuter, et dans notre cas c'est le Run_Load créé précédemment.



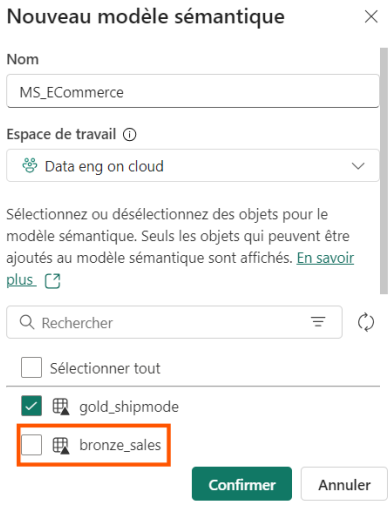

Ainsi à chaque fois qu'un fichier est ajouté, nous n'aurons qu'à exécuter ce pipeline et tous les traitements se feront sans qu'on intervienne forcément.

## *Modèle Sémantique*

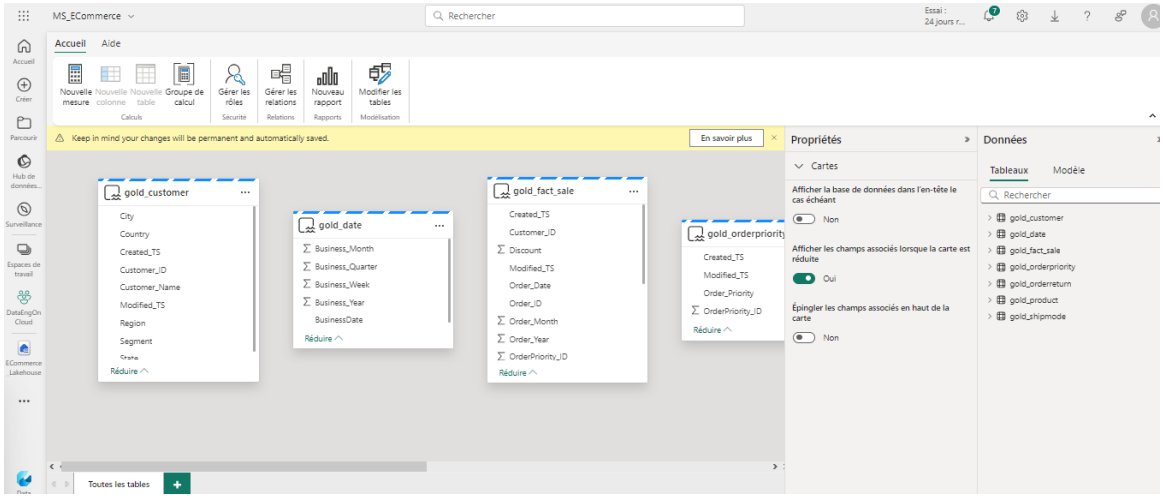
Le modèle sémantique va permettre aux utilisateurs finaux de pouvoir interagir avec des données organisées et nommées de manière compréhensible, sans avoir à connaître la complexité des sources de données sous-jacentes. Il offre des structures de données telles que des tables de faits et des dimensions, des mesures calculées, des colonnes calculées et des hiérarchies. Cela simplifie et accélère l'analyse des données.



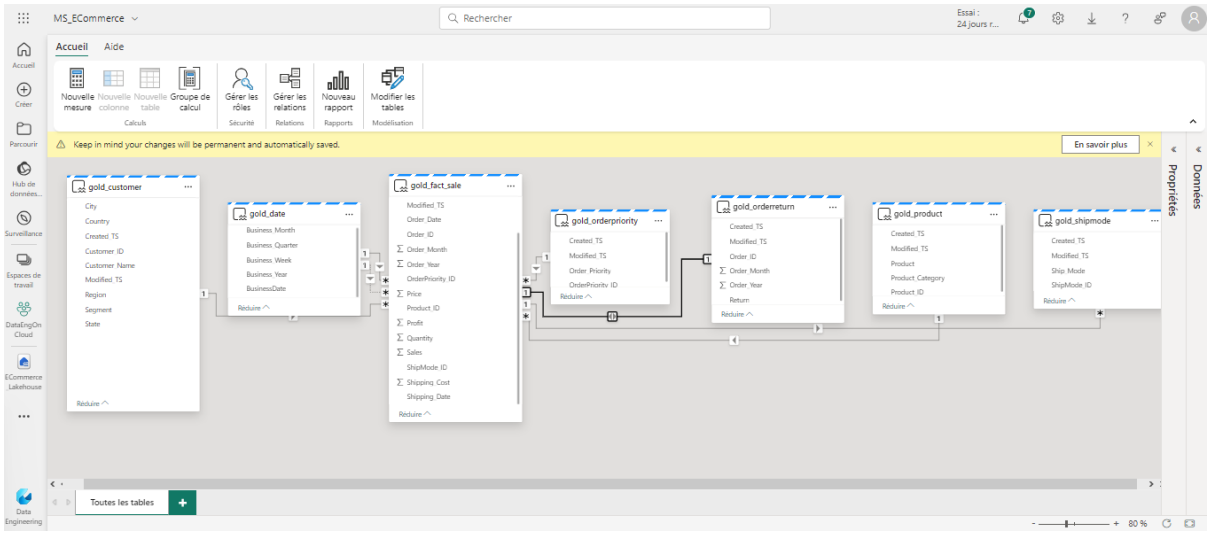

Dans notre modèle sémantique, nous sélectionnons toutes les tables sauf celle qui est en bronze.




Ensuite nous avons une vision globale de notre modèle, ce qui permettra de faire un lien direct entre la table de fait et les tables de dimension qui l'entourent.




Lien entre les tables :

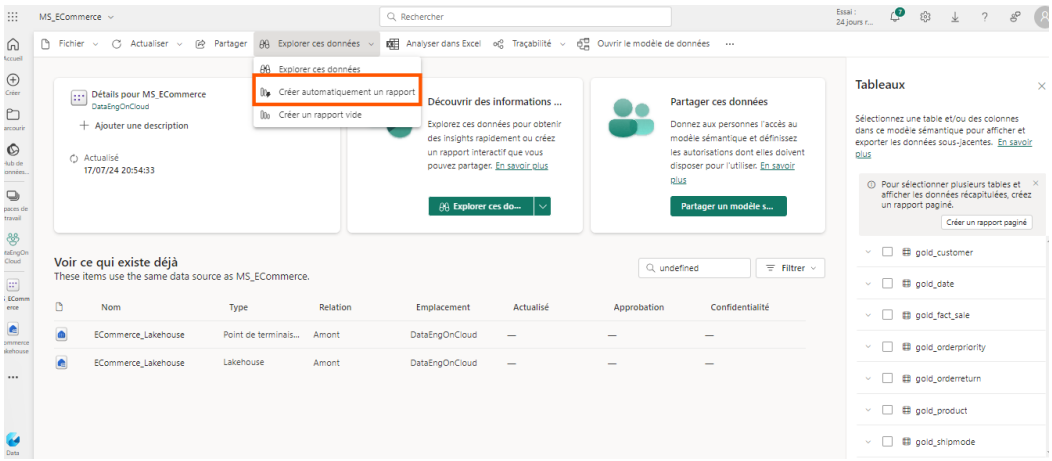



Dans notre projet, une fois tout ceci fait, nous pouvons remarquer que le model a bien était créé et est prêt à être utilisé.

| Nom | Type | Tâche | Propriétaire | Actualisé | Prochaine actualisation | Approbation | Confidentialité | Inclus dans l'application |
|---|---|---|---|---|---|---|---|---|
| Gold_Fact_Sale | Bloc-notes | — | Ndoumbé SOW | — | — | — | — | |
| Gold_OrderPriority | Bloc-notes | — | Ndoumbé SOW | — | — | — | — | |
| Gold_OrderReturn | Bloc-notes | — | Ndoumbé SOW | — | — | — | — | |
| Gold_Product | Bloc-notes | — | Ndoumbé SOW | — | — | — | — | |
| Gold_ShipMode | Bloc-notes | — | Ndoumbé SOW | — | — | — | — | |
| MS_ECommerce | Modèle sémanti... | — | DataEngOnCloud | 17/07/24 20:54:33 | Non applicable | — | — | |
| pip_ecommerce_Load | Pipeline de don... | — | Ndoumbé SOW | — | — | — | — | |
| Run_Load | Bloc-notes | — | Ndoumbé SOW | — | — | — | — | |

## Visualisation

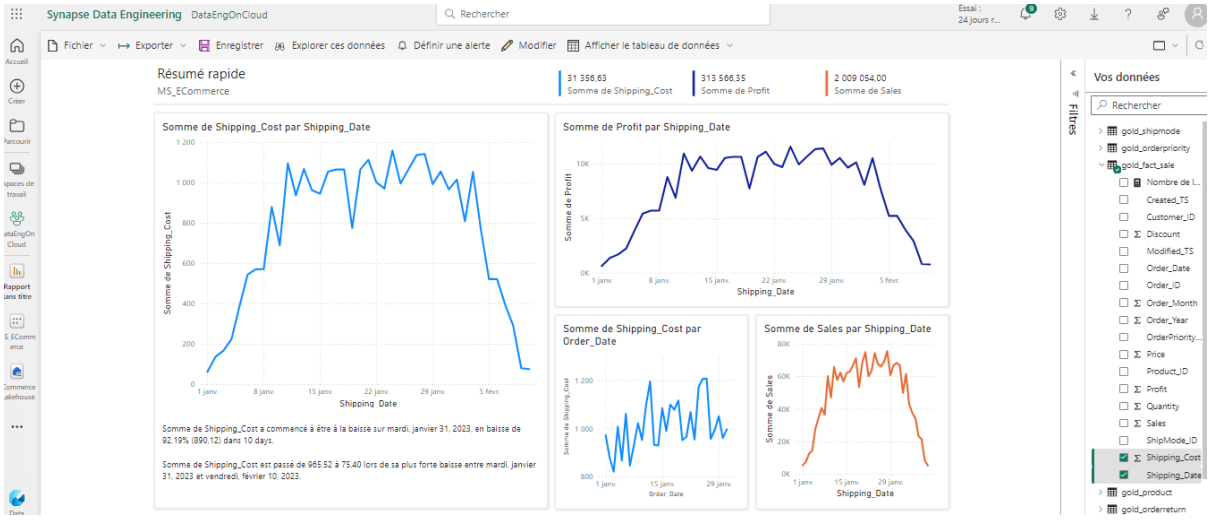
Microsoft Fabric nous offre la possibilité de créer des rapports de manière automatique. Ces rapports sont très pertinents et facilite la prise de décision.



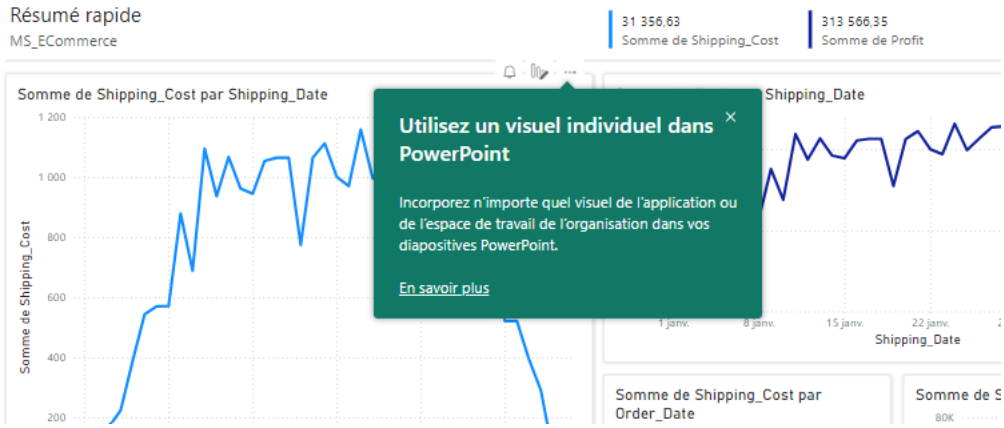

Resultats:

Ici nous pouvons voir de manière très claire l'évolution de la somme de la livraison entre janvier et février. Un commentaire de cette visualisation est fait tout en bas pour mieux expliquer ce que l'on peut voir.

La somme Total de livraison ainsi que ceux du profit et de vente sont tout en haut permettant d'avoir une idée des sommes de manière générale.




Il est également possible de fournir une présentation en Powerpoint des rapports pour une meilleure transparence.



# Conclusion

Ce projet de data engineering sur l'e-commerce avec Microsoft Fabric nous a permis de comprendre l'efficacité d'une architecture de données bien structurée pour une meilleure analyse.

En s'appuyant sur les couches Bronze et Gold, nous avons pu nettoyer et transformer les données de manière optimale, garantissant leur qualité et leur fiabilité. L'élaboration d'un modèle sémantique nous a permis de structurer les données de manière logique, facilitant ainsi l'analyse et l'interprétation des résultats.

Enfin, la visualisation des données a offert une vue d’ensemble claire et intuitive des performances commerciales, mettant en évidence des tendances clé pour la prise de décision.

Ce projet nous a également permis de découvrir Microsoft Fabric et de le manipuler pour mieux le comprendre et l’utiliser dans nos projets futurs.

## Sources :

[1] : https://learn.microsoft.com/fr-fr/fabric/get-started/microsoft-fabric-overview